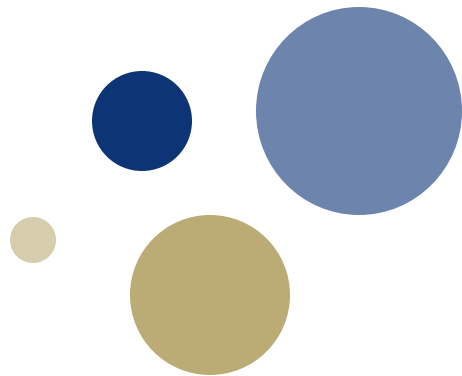




NTNU

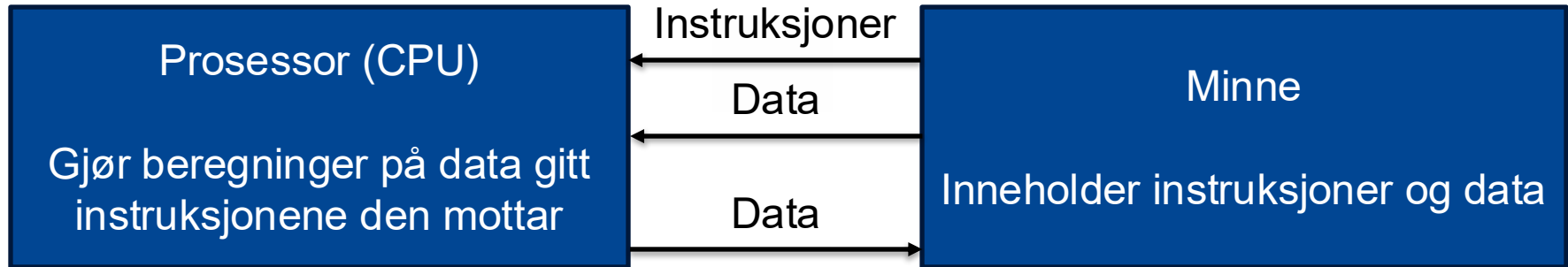
Kunnskap for en bedre verden



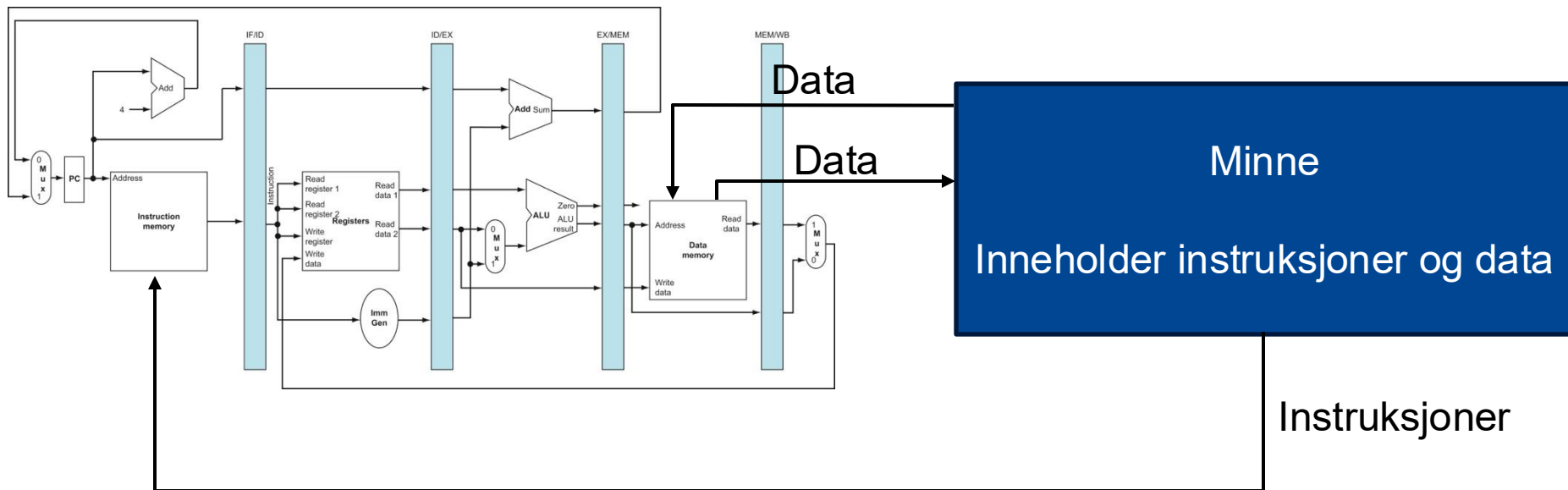
Forelesning 11: Virtuelt minne og parallele datamaskiner

TDT4160 Datamaskiner

Prinsippet om lagrede program



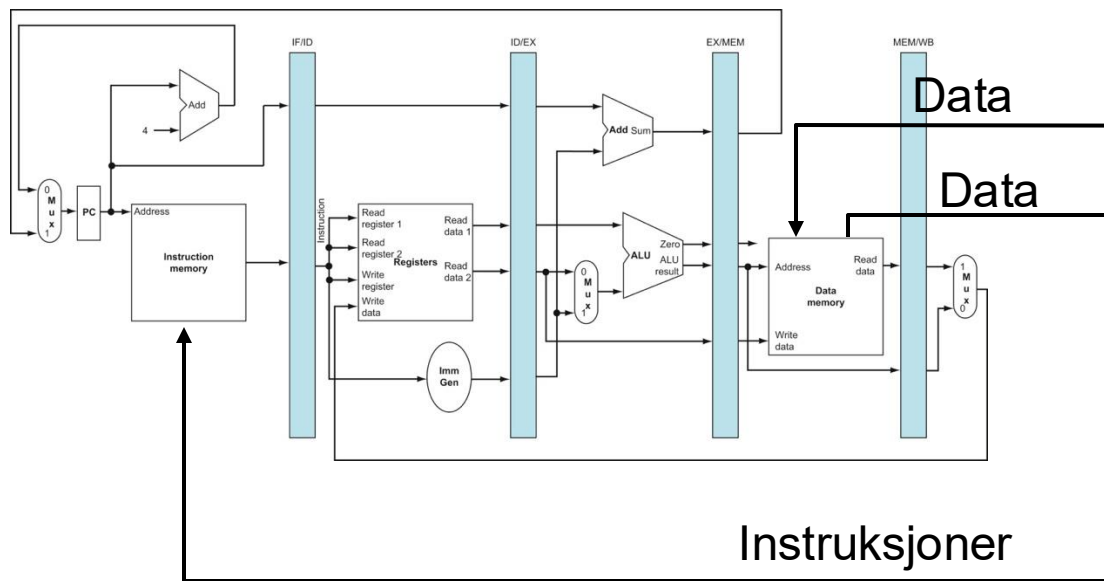
Prinsippet om lagrede program



Prinsippet om lagrede program

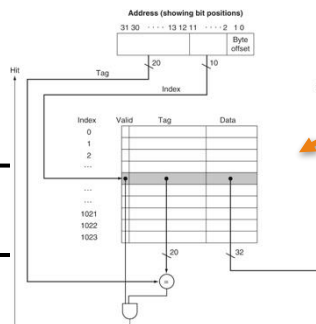
Prosessorbrikken

Prossessor

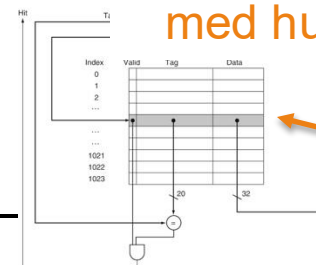


I dag: Virtuelt minne og
parallele datamaskiner

Datahurtigbuffer
(SRAM)



Typisk flere nivåer
med hurtigbuffer



Minne
(DRAM)

Instruksjons-
hurtigbuffer
(SRAM)



Det jeg ikke rakk forrige uke...

Tema 6.1: Minnehierarki og hurtigbuffer (Kapittel 5.3 og 5.4)

HURTIGBUFFER: HØYERE YTELSE

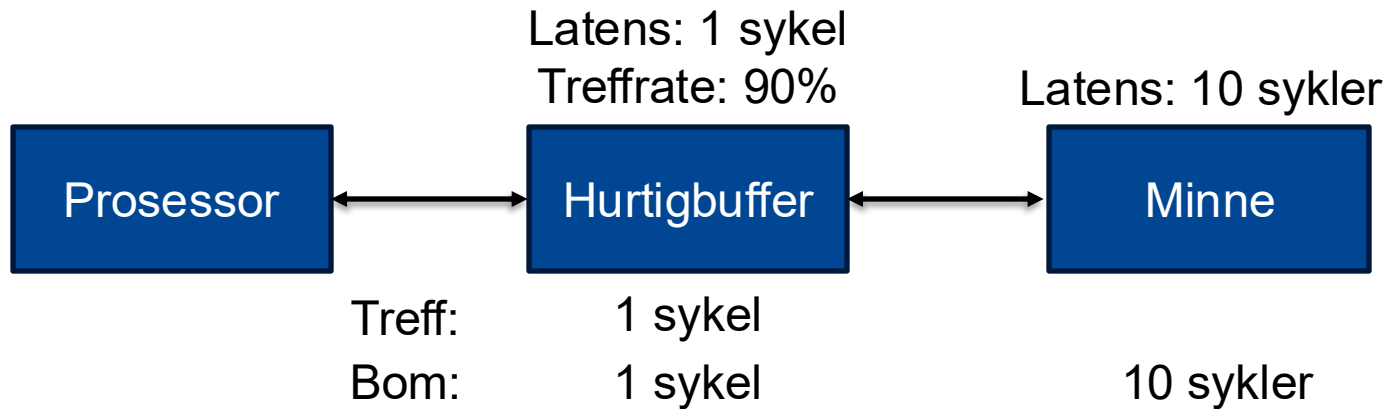
Læringsutbytte T6.1

- Studenten skal kunne forklare begrepene lokalitet i tid og lokalitet i rom og vite hvorfor lokalitet oppstår.
- Studenten skal forstå hvordan og hvorfor et hierarki av minner kan gi illusjonen av ett stort og raskt minne.
- Studenten skal forstå hvordan vi konstruerer et direktetilordnet hurtigbuffer (kunne gjengi og forklare figur 5.10) samt hvordan det kan utvides til å håndtere hurtigbufferblokker som består av mer enn ett ord.
- Studenten kunne forklare hvordan hurtigbuffer kan integreres i en samlebåndsarkitektur.
- Studenten skal kjenne til prinsippene for hvordan hurtigbuffer håndterer skriveoperasjoner.
- Studenten skal kunne beregne hvordan minneaksesstid påvirker datamaskinens ytelse.
- **Studenten skal kunne beregne hvordan treffraten i flernivå hurtigbuffer påvirker den gjennomsnittlige aksesstiden.**
- Studenten skal forstå hvorfor settassosiative og fullassosiative hurtigbuffer kan øke treffraten og hvordan de konstrueres (gjengi og forklar figur 5.18).
- Studenten skal kjenne til hvordan man på programvarenivå kan påvirke treffraten i hurtigbuffer.

Repetisjon: Minnelatens

$$CPI = \frac{C_{Beregning}}{I_{Total}} + \frac{C_{Minne}}{I_{Total}}$$

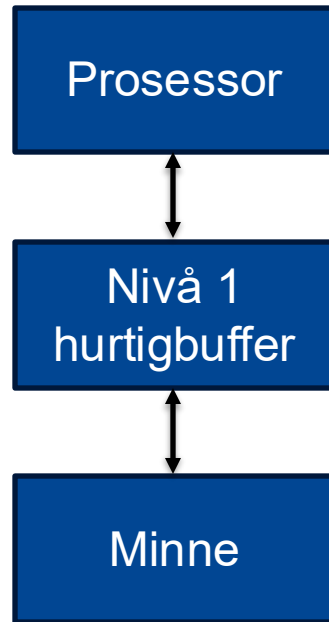
- **Minnelatens** = Antallet klokkesykler det tar å hente en verdi fra en minneenhet
- Vi kan dermed se på C_{Minne}/I_{Total} som den gjennomsnittlige minnelatensen over alle instruksjoner
- Vi kjenner antallet instruksjoner, så det er C_{Minne} vi må måle/estimere
 - Hvis vi deler C_{Minne} på antall minneforespørsler får vi «Average Memory Access Latency (AMAT)»



$$AMAT = 0.9 \cdot 1 + 0.1 \cdot (1 + 10) = 2$$

Flernivå hurtigbuffer

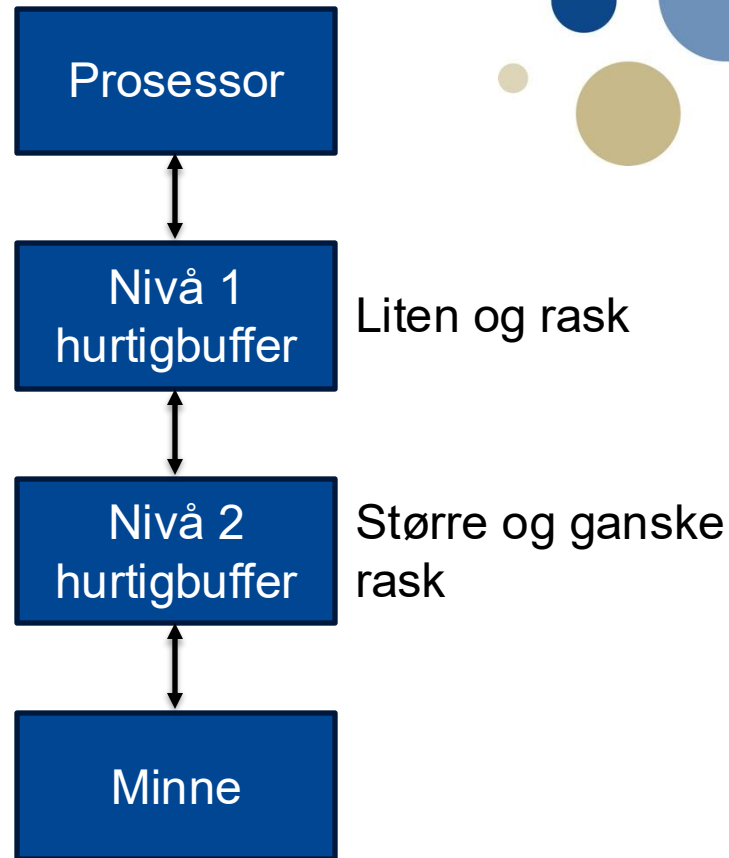
- Trefflatensen til et hurtigbuffer øker med størrelsen på hurtigbufferet
 - ... men aksess til hurtigbufferene i prosessoren er ofte på kritisk sti
 - ... og derfor må de være relativt små (for eksempel 32 KB)
- Løsning: Flere nivåer med hurtigbuffer
 - Reduserer latensen til bom fordi noen bom nå treffer i Nivå-2 hurtigbufferet



Flernivå hurtigbuffer

- Trefflatensen til et hurtigbuffer øker med størrelsen på hurtigbufferet
 - ... men aksess til hurtigbufferene i prosessoren er ofte på kritisk sti
 - ... og derfor må de være relativt små (for eksempel 32 KB)
- Løsning: Flere nivåer med hurtigbuffer
 - Reduserer latensen til bom fordi noen bom nå treffer i Nivå-2 hurtigbufferet

Menti!



Eksempel: Aksestidsberegning

Oppgave 1

Nivå	Andel treff	Latens	Bidrag

Oppgave 2

Nivå	Andel treff	Latens	Bidrag

Eksempel: Aksesstidsberegning



Oppgave 3

Nivå	Andel treff	Latens	Bidrag



Tema 6.3: Virtuelt minne (Kapittel 5.6 og 5.7)

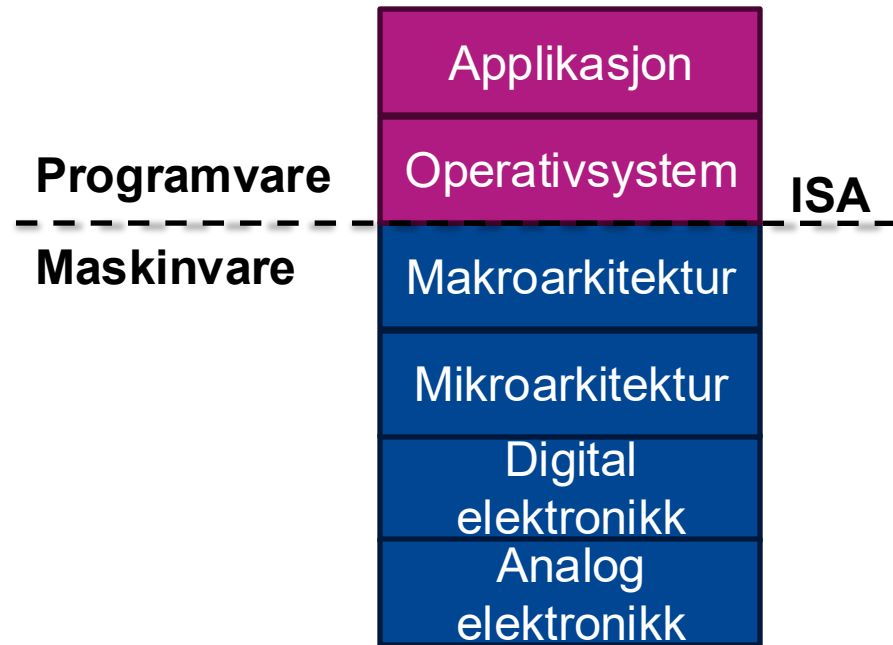
VIRTUELT MINNE

Læringsutbytte T6.3

- Studenten skal kunne forklare hvordan **virtuelle adresser oversettes til fysiske adresser**, inkludert arbeidsfordelingen mellom maskinvare og programvare.
- Studenten skal kunne forklare hvordan et «**Translation Lookaside Buffer (TLB)**» brukes til å implementere rask adresseoversettelse.
- Studenten skal kunne forklare hvordan virtuelt minne kan brukes til å **beskytte prosesser fra hverandres minneaksesser**.
- Studenten skal kunne forklare hvordan **sidefeil** håndteres.
- Studenten skal kunne forklare hva en **virtuell maskin** er.

Abstraksjonslag i datamaskinen

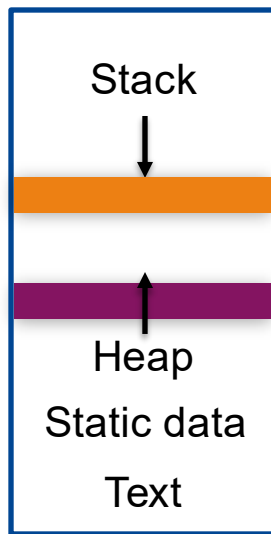
- Så langt har vi nesten utelukkende hatt et maskinvareperspektiv
 - Vi har enten løst rene maskinvareproblemer
 - ... eller vi har fått programvare til å gjøre jobben vår enklere (for eksempel kompilatoren)
- Datamaskinen må også bidra til å løse problemer på programvarenivå
 - Eksempel: Presise avbrudd
 - Nå: Virtuelt minne (og virtuelle maskiner)



Datamaskiner kjører typisk flere programmer samtidig

- ... men i våre prosessorer kan alle programmer skrive hvor som helst i minnet
 - Dette er både upraktisk og usikkert
- Løsning: Virtuelt minne
 - Operativsystemet bestemmer over det fysiske minnet
- Vi må oversette hver bidige minneadresse
 - ... som vil bli alt for tregt uten maskinvarestøtte

Programmets adresserom
(virtuelle adresser)



Adresseoversettelse

Datamaskinens minne
(fysiske adresser)



Sider og sidetabellen

- Vi deler inn minnet i sider («pages») (typisk på 4KiB)
- Operativsystemet bruker **sidetabellen** til å tilordne virtuelle sider til fysiske sider
 - Sidetabellen gir også informasjon om hvilke prosesser som har lov til å skrive og/eller lese denne siden
 - **Prosess** = Et kjørende program med tilstand (for eksempel PC)
 - Sidetabellen er lagret i minnet og kan bare endres av operativsystemet

Programmets adresserom
(virtuelle adresser)



Datamaskinens minne
(fysiske adresser)



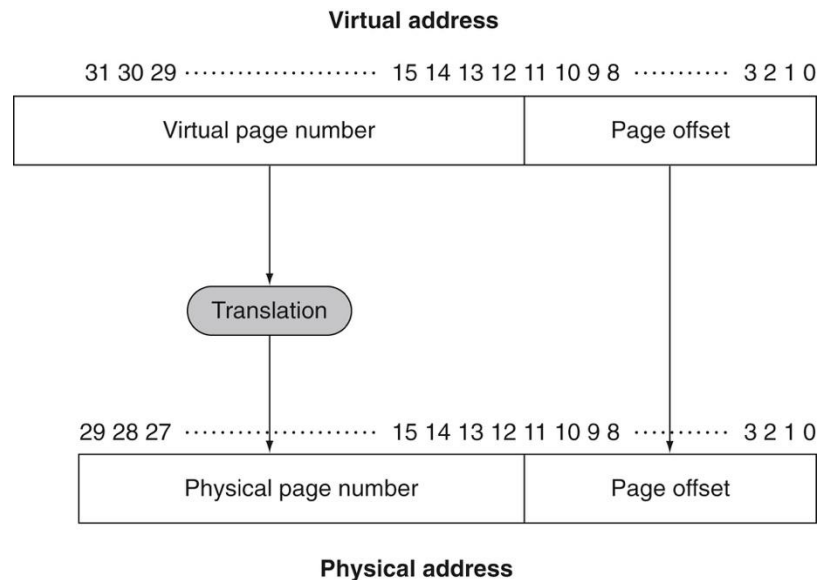
Sidetabell

v2 → f1

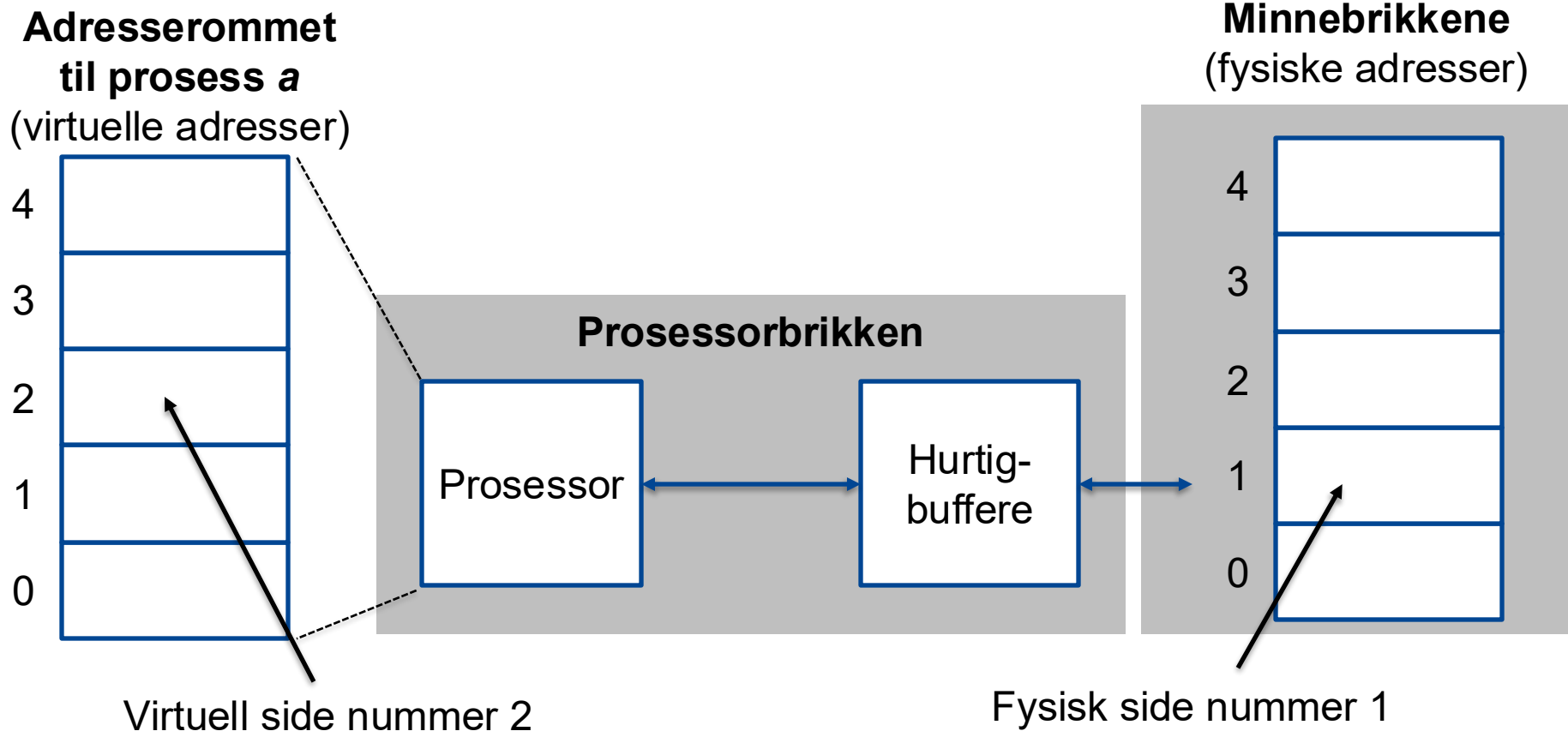
v1 → f2

Adresseoversettelse

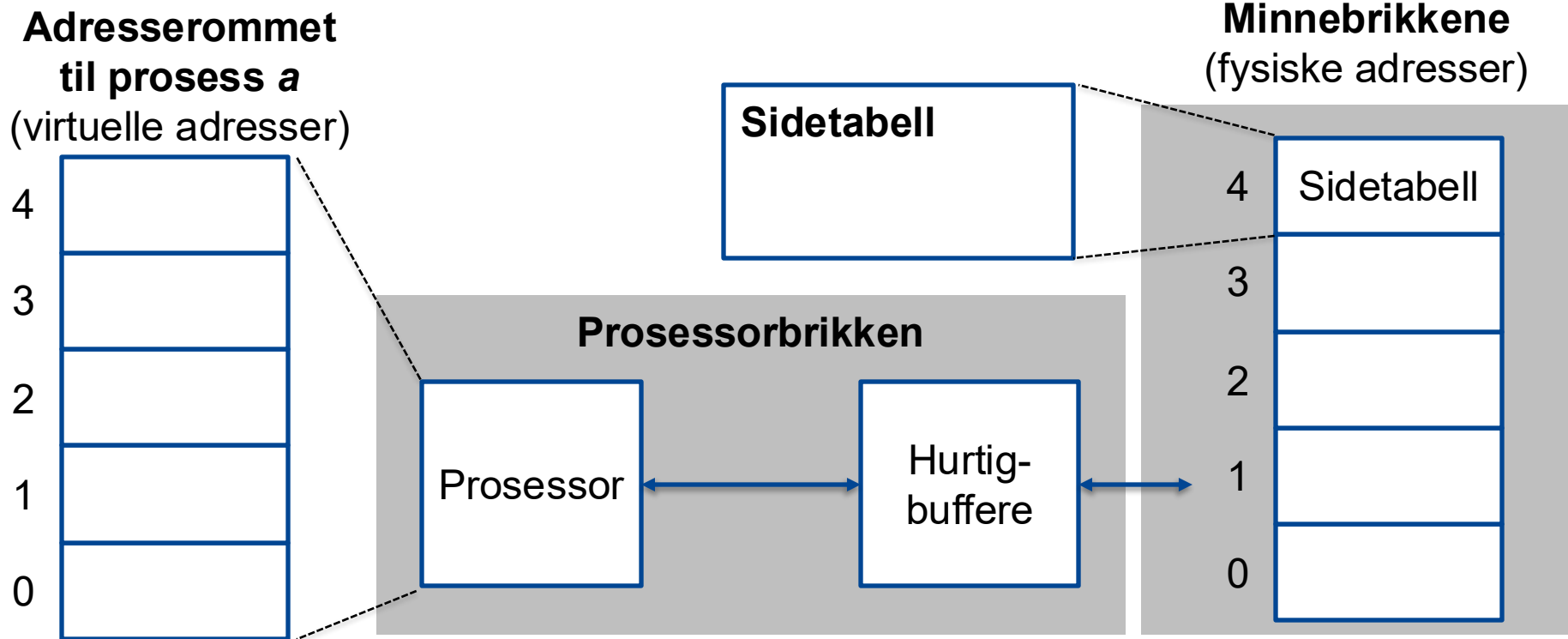
- Akkurat som med byte offset i hurtigbuffer vil de minst signifikante bitene i adressen gå til verdier inni siden
 - Kalles «page offset»
- Resten av adressen angir sidennummeret
 - ... og når vi oversetter adresser bytter vi ut det virtuelle sidennummeret med det fysiske sidennummeret gitt i sidetabellen



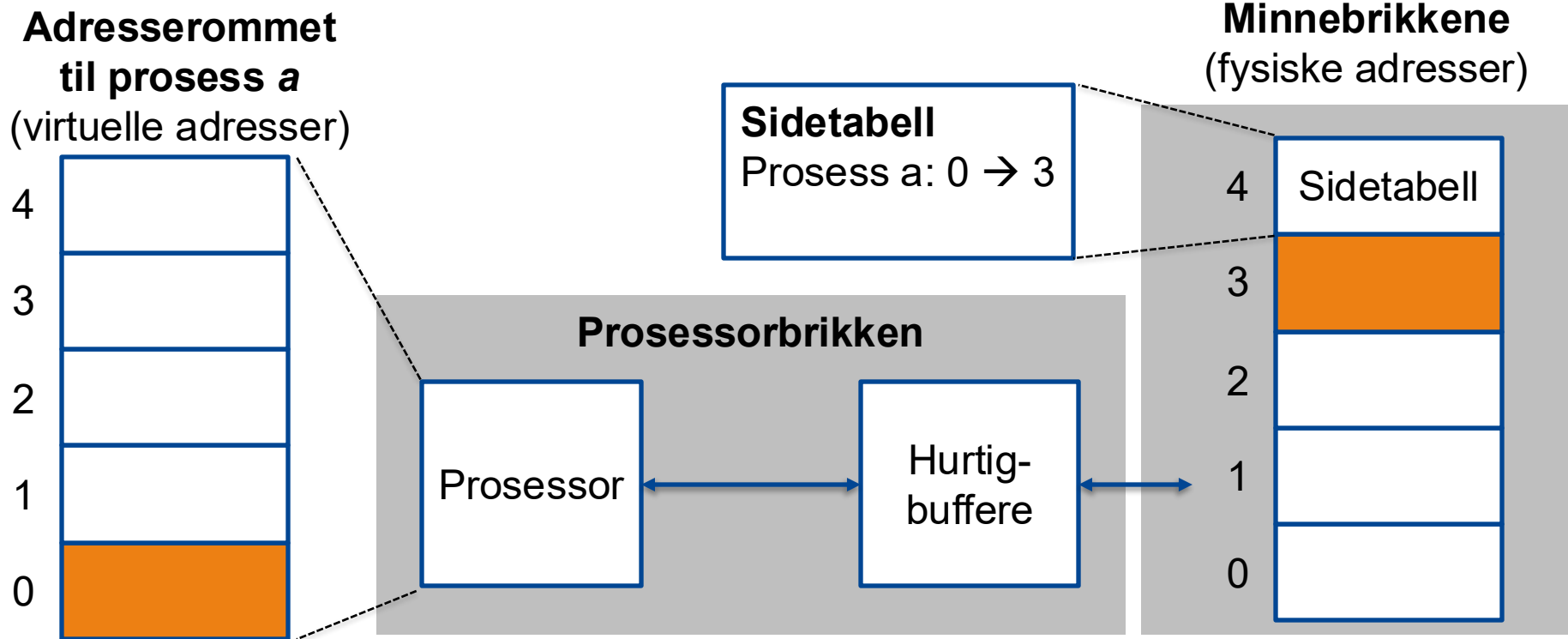
Minnesystem med virtuelt minne



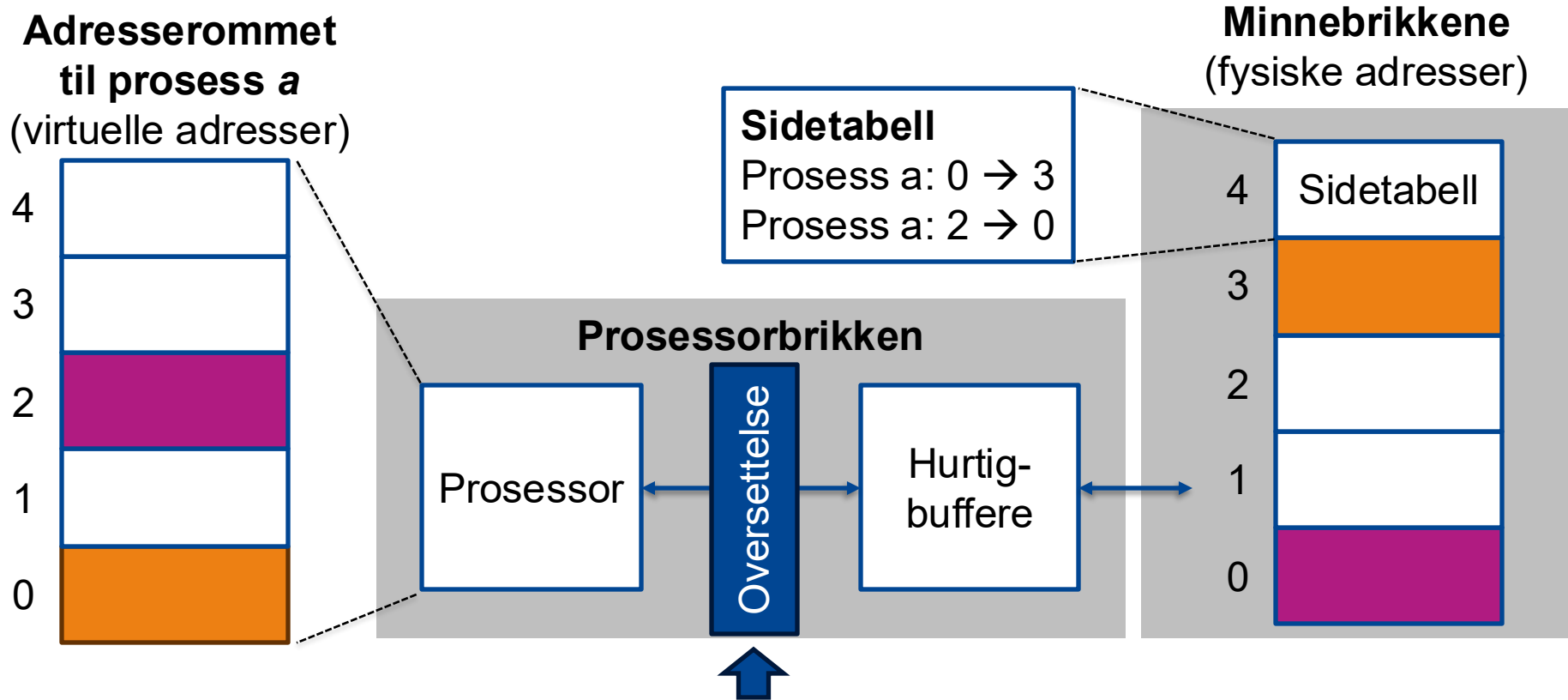
Minnesystem med virtuelt minne



Minnesystem med virtuelt minne

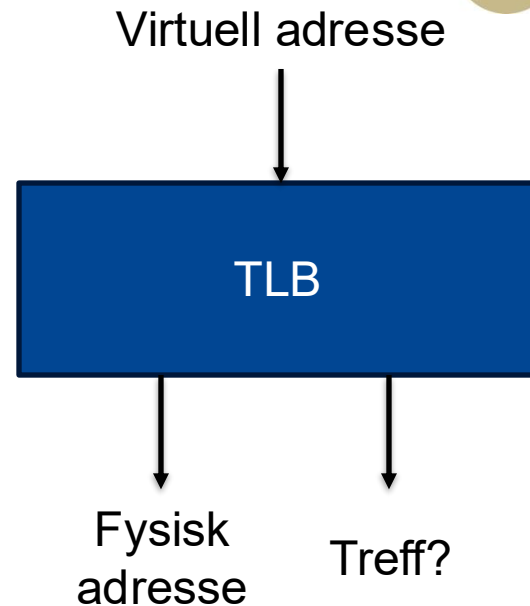


Minnesystem med virtuelt minne



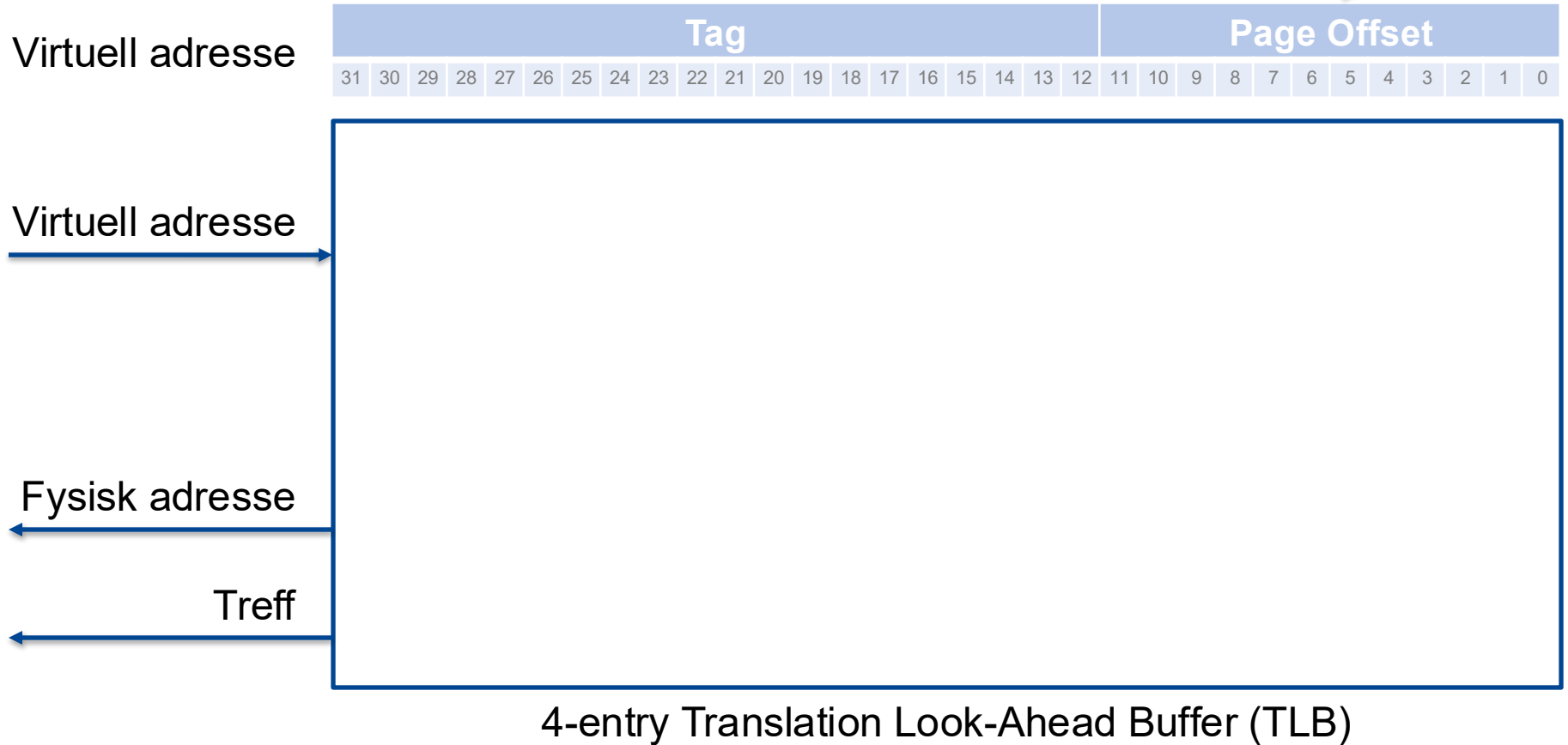
Translation Lookaside Buffer (TLB)

- Løsning: Et hurtigbuffer for adresseoversettelser
 - Kalles av historiske årsaker «Translation Lookaside Buffer (TLB)»
 - Vi har typisk en TLB for instruksjoner og en for data (for å ikke lage en strukturfare)
- Bom i TLB er dyrt
 - ... fordi vi må hente oversettelsen fra sidetabellen
 - Vi må også håndtere bom i TLB før vi kan starte minneforespørselen
 - ... fordi ellers vet vi ikke hvilken adresse vi skal aksessere
- Optimalisering
 - 1: Gjør TLBen fullassosiativ [øker treffrate]
 - 2: Legg til maskinvarestøtte for å slå opp i sidetabellen («Page table walker») [reduserer kostnad ved bom]



Eksempel: Fullassociativ TLB

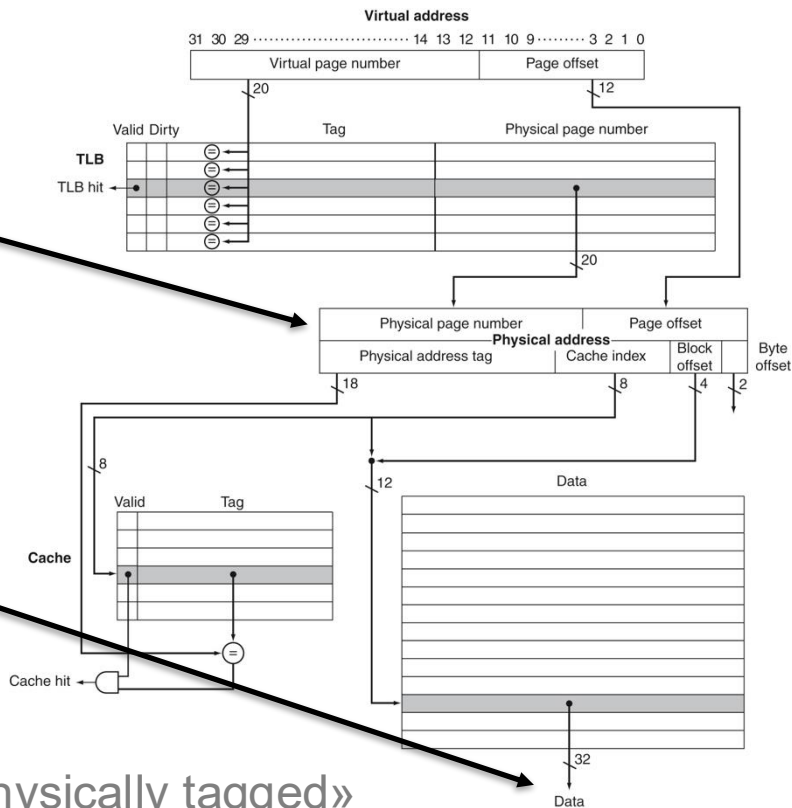
4 KiB sider
($2^{12} = 4096$)



Eksempel: Intrinsity FastMATH TLB

Vi slår opp i TLBen først for å få tak i den fysiske adressen

... så slår vi opp i hurtigbufferet for å få tak i verdien



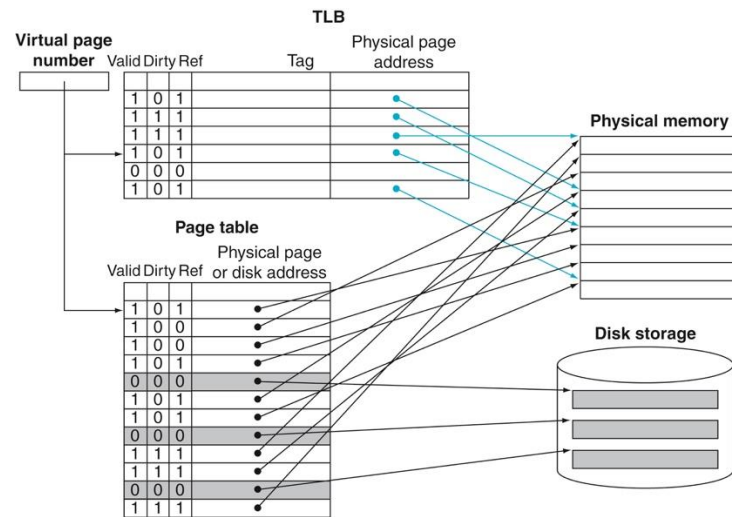
Optimalisering: «Virtually indexed, physically tagged»
→ gjør det mulig å slå opp i TLB og hurtigbufferet i parallel

Beskyttelse («Protection»)

- Vi må isolere minneområdene til prosessene fra hverandre (feil og villet handling)
- Vi trenger følgende funksjonalitet for å implementere beskyttelse:
 - (Minst) to systemmodi
 - **Brukermodus** («user mode») og **systemmodus** («system mode» eller «kernel mode»)
 - **Privilegerte instruksjoner**: Genererer unntak hvis de kjøres i brukermodus
 - Tilstand i prosessoren som kan leses i brukermodus men kun skrives i systemmodus
 - Modusbit: Er vi i systemmodus eller ikke?
 - Sidetabellpeker: Peker til sidetabellen til prosessen som kjører akkurat nå
 - TLBene
 - Kunne bytte mellom brukermodus og systemmodus
 - Brukermodus til systemmodus: **Systemkall** («System call»), instruksjonen `ecall` i RISC-V
 - Systemmodus til brukermodus: **Systemretur**, instruksjonen `sret` i RISC-V
- Bytte mellom prosesser kalles et **kontekstbytte** («context switch»)
 - Operativsystemet lagrer all tilstanden til prosessen slik at den kan starte igjen som om ingenting har skjedd (PC, registerverdier, sidetabell, etc.) → *Tenk: Funksjonskall for prosesser*
 - Operativsystemet er en privilegert prosess (en prosess som kan gjøre ting som vanlige prosesser ikke kan)

Hvordan oppdateres sidetabellen?

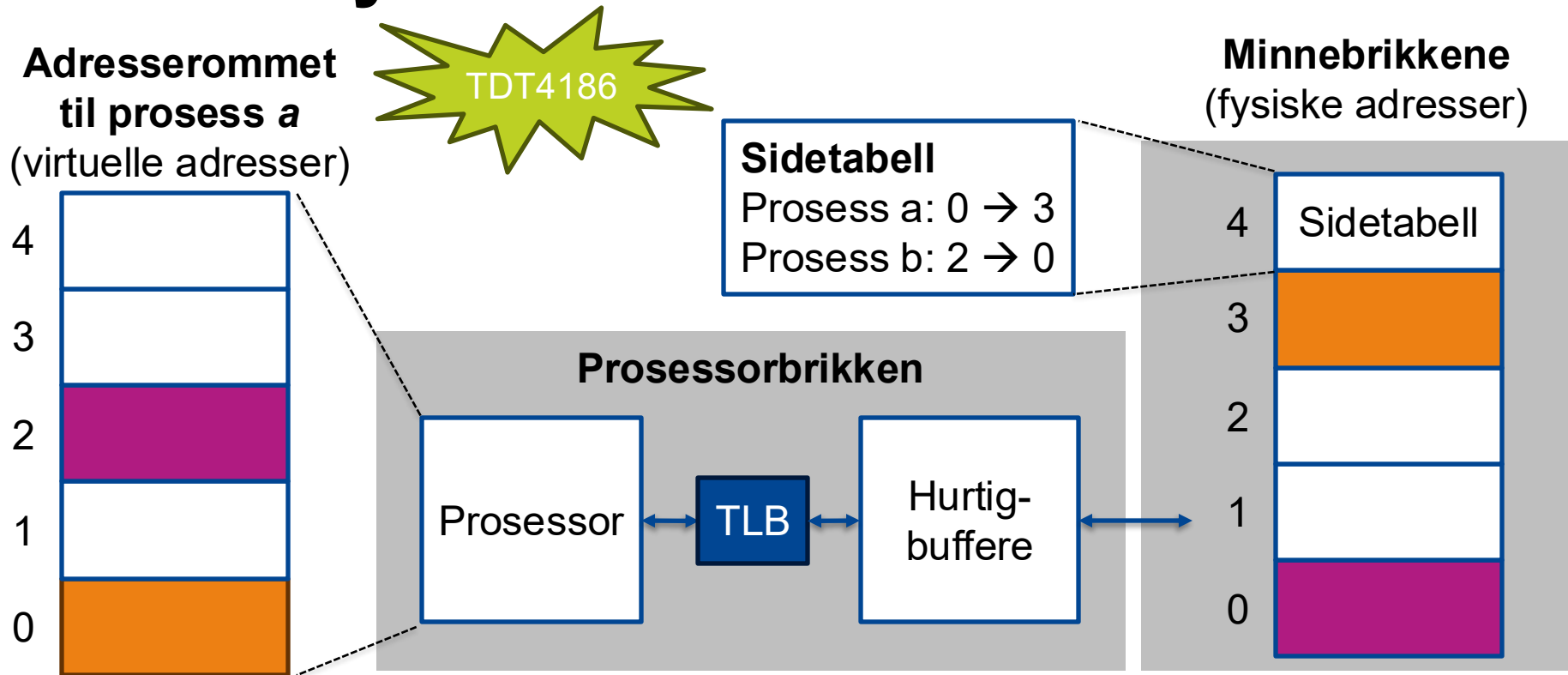
- **Sidefeil** oppstår når siden vi aksesserer ikke er i minnet
 - «Swap space»: Et sted på disk/Flash der operativsystemet kan lagre sidene til prosesser
 - Sider legges typisk i minnet når de blir aksessert (og ikke når de blir allokert)
- Prosess *a* utfører en load-instruksjon men siden er ikke i sidetabellen → **en sidefeil** («*page fault*»)
 1. Prosessoren fullfører alle instruksjoner *som er eldre enn load-instruksjonen* og generer et **unntak**
 - Adressen i PC peker på load-instruksjonen
 2. Operativsystemet håndterer **unntaket**
 - a. Bytter til systemmodus og lagrer **tilstanden** til Prosess *a* (inkludert PC og registrene x1 til x31)
 - b. Setter inn den manglende siden i minnet og oppdaterer sidetabellen
 - c. Legger inn den nye tilordningen i TLBen
 - d. Skriver tilbake **tilstanden** til Prosess *a*
 - e. Bytter til brukermodus og returner kontroll til Prosess *a*
 3. Prosessoren utfører load-instruksjonen
 - Den vil nå treffe i TLBen!



Sidefeil er kjempedyrt
(tar typisk tusenvis av klokkesyker)

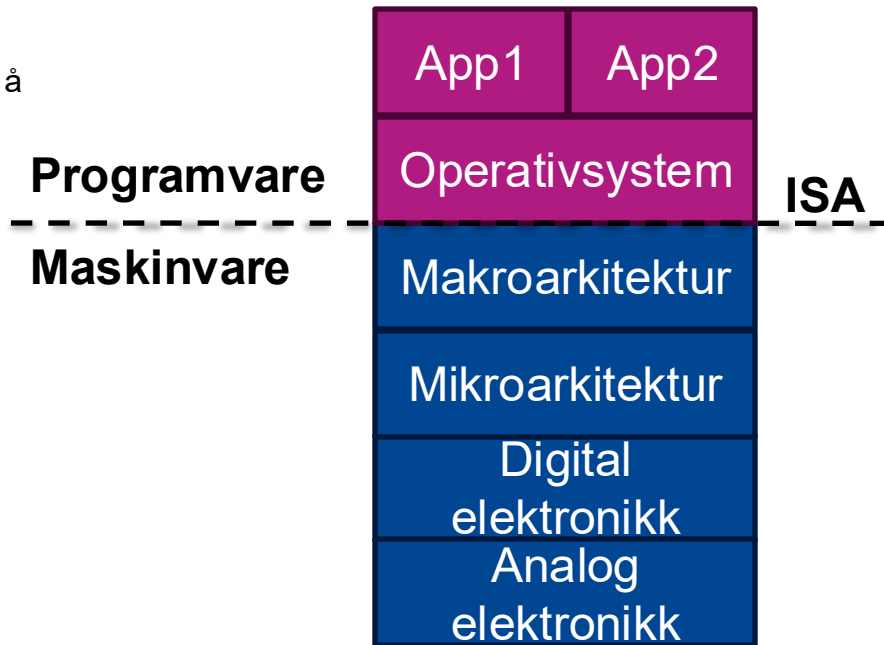
- Dette ville vært mye vanskeligere uten **presise unntak**!

Minnesystem med virtuelt minne



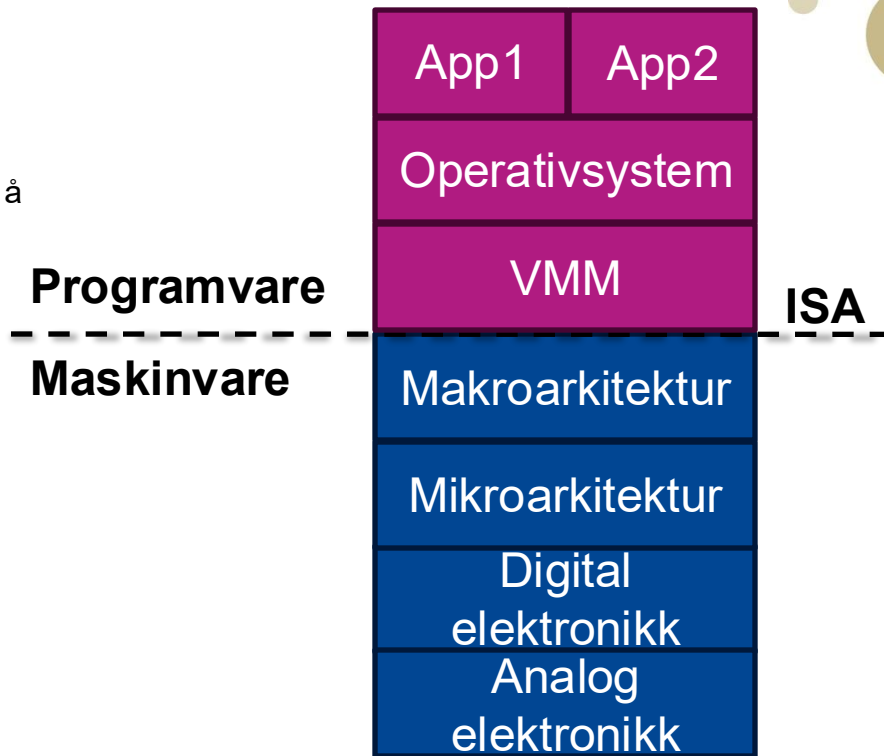
Virtuelle maskiner

- Vi har nå et system der hver prosess opplever at den kjører alene på maskinen
 - Operativsystemet er en privilegert prosess
 - ... og vi kan dermed utvide abstraksjonen til å kjøre flere operativsystemer på samme maskin og dermed lage **virtuelle maskiner!**



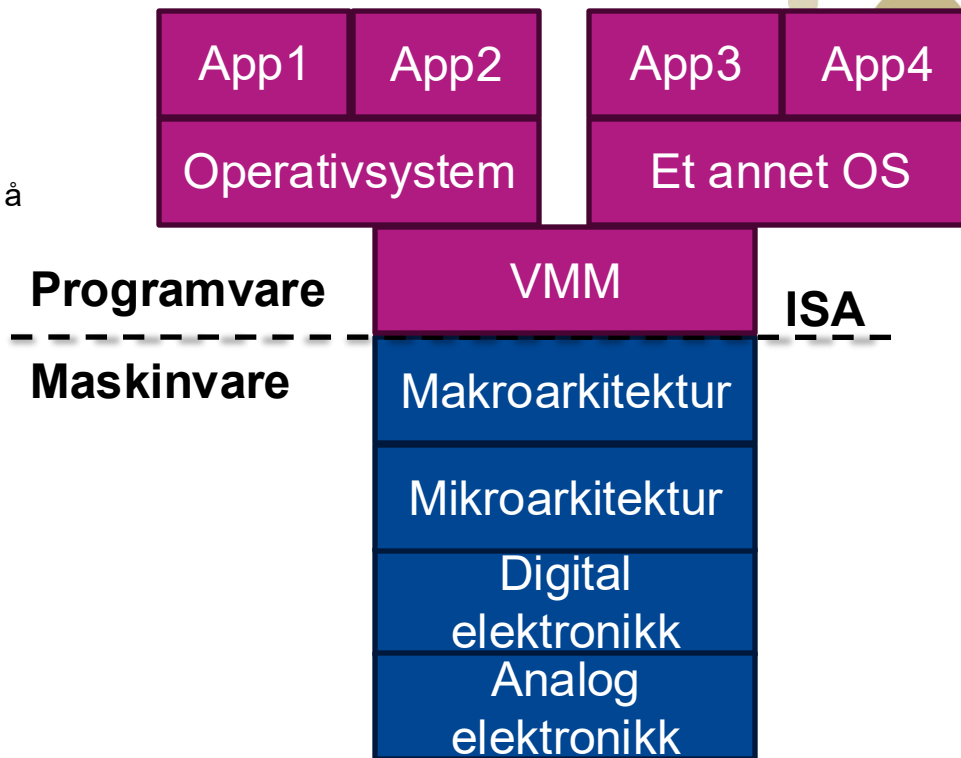
Virtuelle maskiner

- Vi har nå et system der hver prosess opplever at den kjører alene på maskinen
 - Operativsystemet er en privilegert prosess
 - ... og vi kan dermed utvide abstraksjonen til å kjøre flere operativsystemer på samme maskin og dermed lage **virtuelle maskiner!**
- Nytt abstraksjonslag: «Virtual Machine Monitor (VMM)» eller «hypervisor»



Virtuelle maskiner

- Vi har nå et system der hver prosess opplever at den kjører alene på maskinen
 - Operativsystemet er en privilegert prosess
 - ... og vi kan dermed utvide abstraksjonen til å kjøre flere operativsystemer på samme maskin og dermed lage **virtuelle maskiner!**
- Nytt abstraksjonslag: «Virtual Machine Monitor (VMM)» eller «hypervisor»
- Mye brukt i skyinfrastrukturer
 - ... fordi hver kunde kan sette opp sitt programvaremiljø som så kan tilordnes fysiske maskinressurser ved behov (f. eks. Amazon Web Services)

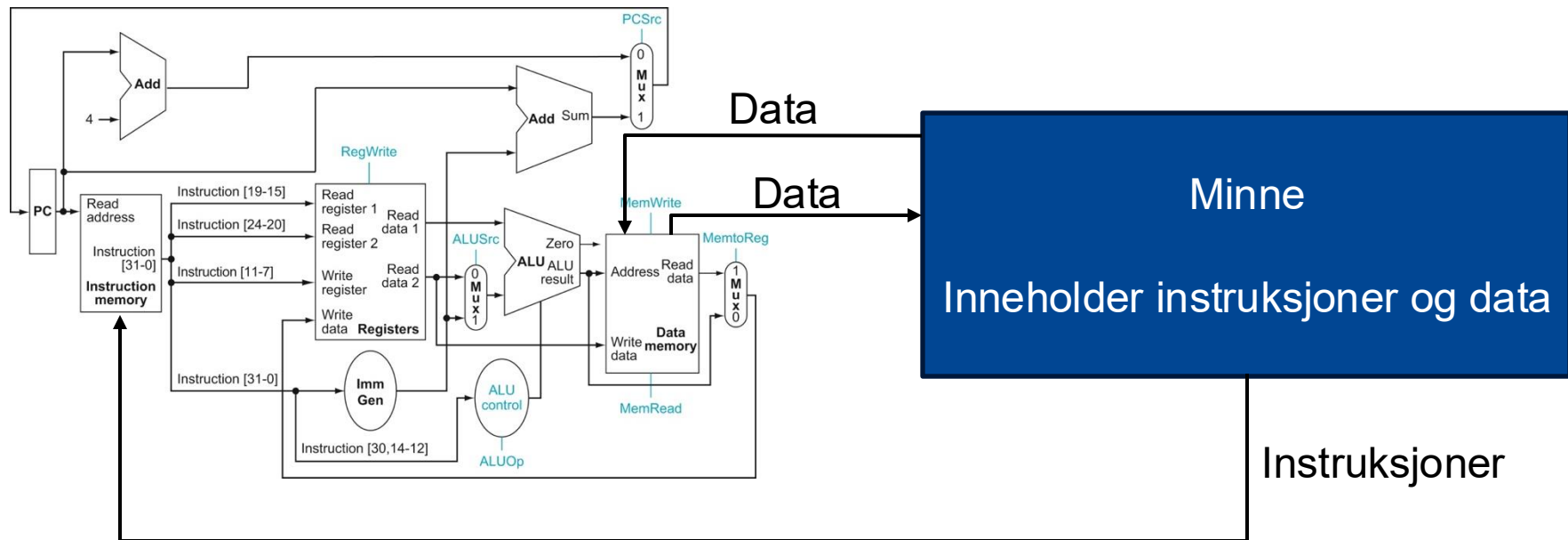




Oppsummering

PROSESSOR OG MINNESYSTEM

Prinsippet om lagrede program

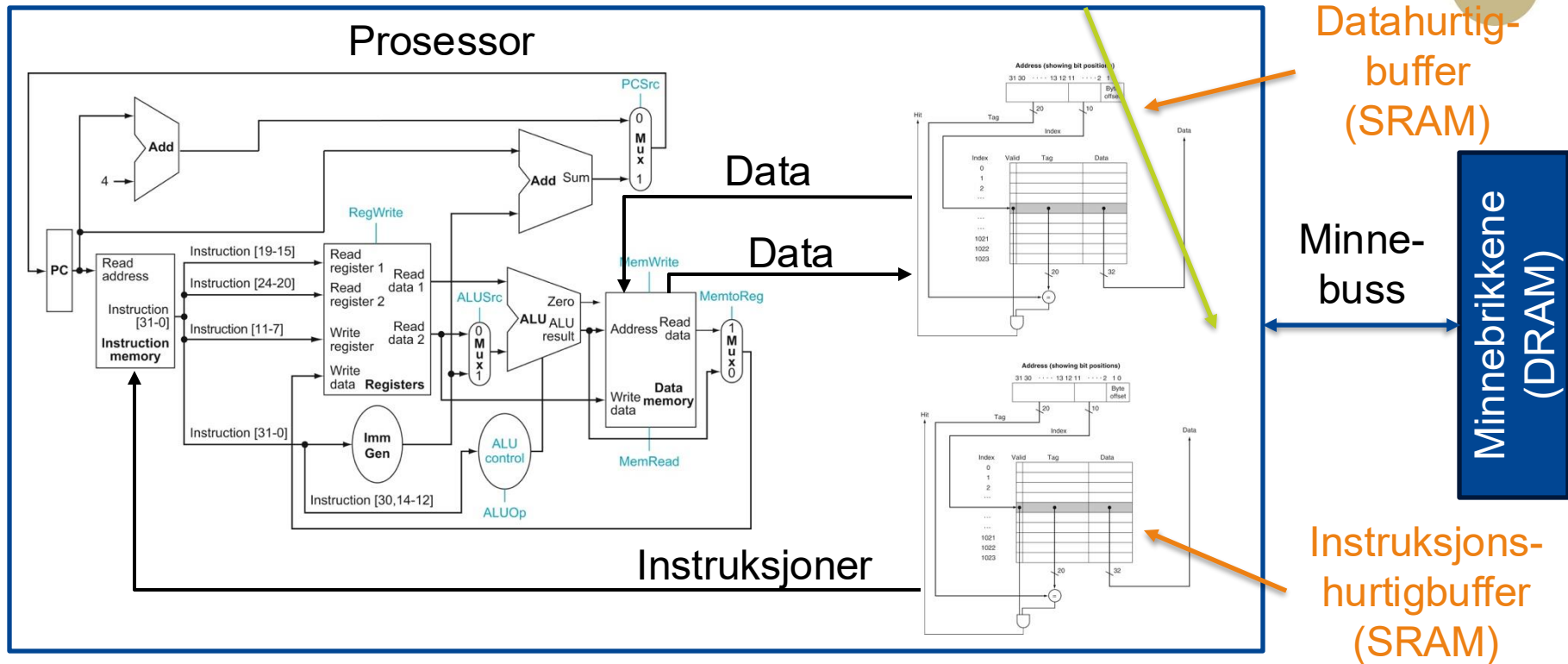


Prinsippet om lagrede program

Magien er borte!

Proessorbrikken

Ofte flere nivåer med hurtigbuffer (SRAM)





Tema 7.1 (Kapittel 6.1, 6.2, 6.3)

YTELSE OG FLYNNNS TAKSONOMI

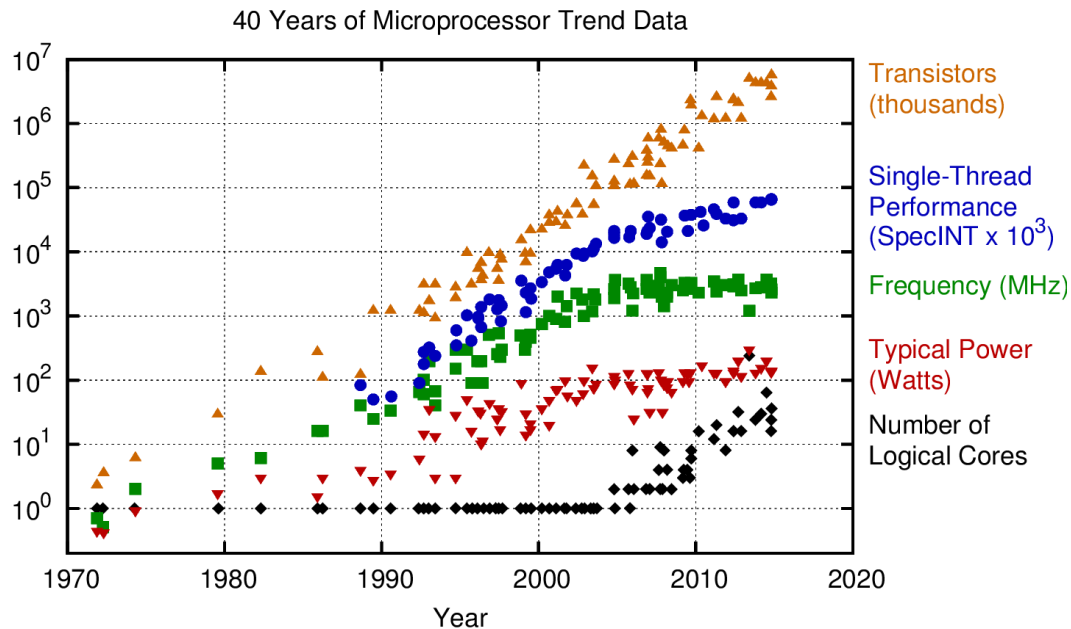
Læringsutbytte T7.1

- Studenten skal kunne forklare hvorfor det er enklere å utnytte **parallelitet** mellom **uavhengige program** enn **innad i ett program**.
- Studenten skal kunne bruke **Amdahls lov** til å analysere **skalerbarheten** til parallelle programmer.
- Studenten skal kjenne til konseptene **sterk og svak skalering** og kunne bruke dem til å vurdere skalerbarheten til ulike applikasjoner.
- Studenten skal kunne forklare **Flynns taksonomi** og kunne gi eksempler på SISD, SIMD, og MIMD maskiner.
- Studenten skal kunne bruke Roofline-modellen og konseptet operasjonsintensitet til å analysere hvorvidt en applikasjon er minnebundet eller beregningsbundet på en gitt datamaskin.

Hvorfor er stort sett alle datamaskiner parallelle?

$$Power \propto 1/2 \times Capacitive\ load \times Voltage^2 \times Frequency\ switched$$

- Dagens prosessorer er begrenset av effektforbruk
 - ... og effektforbruket øker lineært med klokkefrekvensen
- Vi kan ikke (lenger) øke klokkefrekvensen
 - ... og dette begrenser ytelsesøkningen til en enkelt prosessor
- Antallet transistorer på per brikke har imidlertid fortsatt å doble seg hvert andre år (Moore's lov)
 - ... og effektforbruket vokser lineært med kapasitansen til de transistorene som bytter tilstand (som ikke er på langt nær alle)
- Løsning: Vi legger til flere prosessorkjerner
 - Høyere ytelse uten (vesentlig) økt effektforbruk
 - Udfordring: Bedre ytelse krever parallelitet på programvarenivå



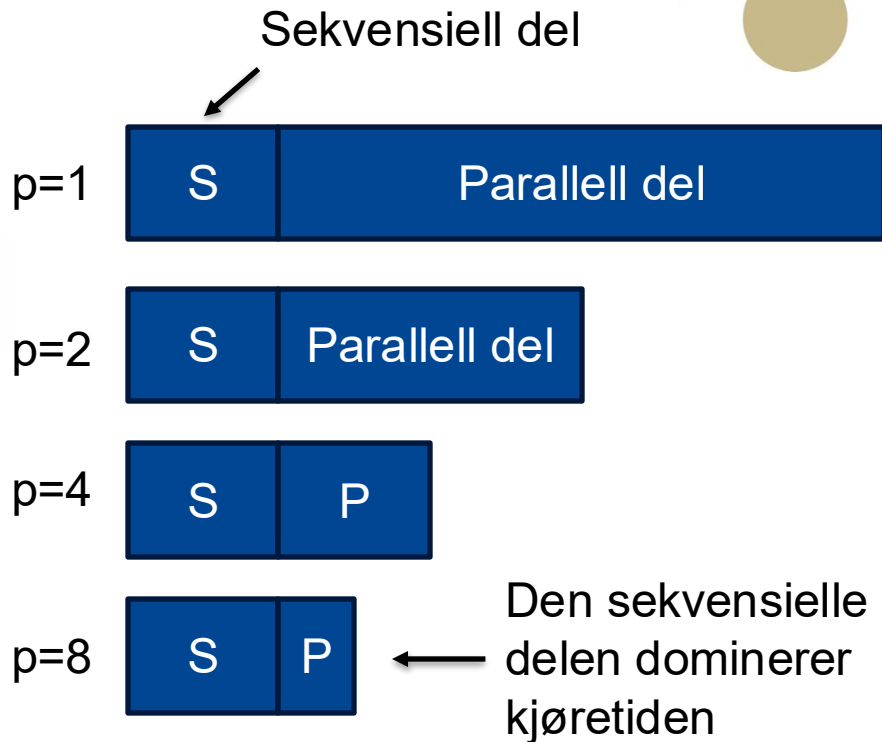
Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2015 by K. Rupp

Parallellitet på programvarenivå

- Vi skiller mellom parallellitet mellom (uavhengige) program og mellom (avhengige) oppgaver i ett program
 - Uavhengige: Programmer som kjøres i parallell (f. eks. «Process-Level Parallelism»)
 - Avhengige: Et parallelt program
- Det er trivielt å utnytte parallellitet mellom uavhengige program
 - Typisk eksempel: Du skal gjøre den samme jobben med n forskjellige innverdier
- Det er vanskelig(ere) å utnytte parallellitet til å løse én oppgave
 - ... fordi vi da må passe på at alle avhengigheter respekteres når vi skriver koden

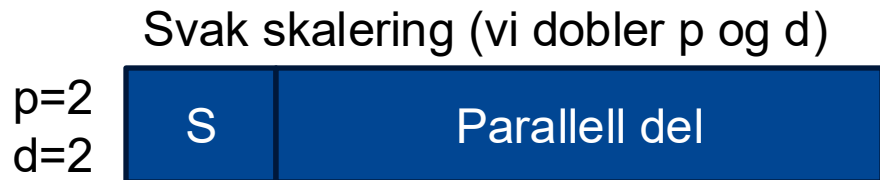
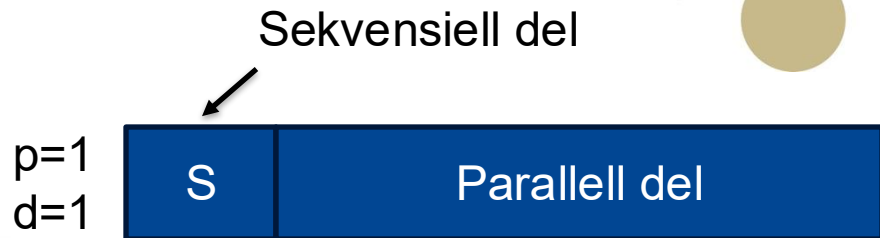
Amdahl's lov for parallelle program

- Amdahl: Et parallelt program består av en sekvensiell og en parallell del
- Når vi øker antall prosessorer
 - ... fordeles kjøretiden til den parallelle delen jevnt utover alle prosessorene
 - ... mens kjøretiden til den sekvensielle delen forblir uendret



Sterk og svak skalering

- Amdahls lov antar at programmet gjør samme mengde arbeid selv om man øker antallet prosessorer
 - Med andre ord: vi beholder samme problemstørrelse
 - Dette kalles sterk skalering («strong scaling»)
- Vi kan også anta at problemstørrelsen vokser i takt med antallet prosessorer
 - Da holder forholdet mellom den sekvensielle og den parallelle delen seg omtrent konstant
 - Dette kalles svak skalering («weak scaling»)
- Hvilket skaleringsmønster som best beskriver et program er ofte gitt av situasjonen det brukes i

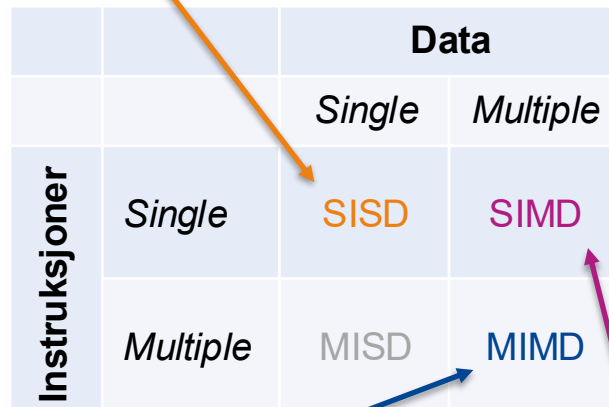


Flynns taksonomi

- Hvilke former for parallelle datamaskiner kan man lage?

Eksempel: En datamaskin med enkelt prosessor

- To dimensjoner:
 - Har vi en (S) eller flere (M) instruksjonsstrømmer (I)?
 - Har vi en (S) eller flere (M) datastrømmer (D)?



The diagram shows a 2x2 matrix for Flynn's taxonomy. The columns are labeled 'Data' (Single, Multiple) and the rows are labeled 'Instruksjoner' (Single, Multiple). The cells contain the following terms: SISD (Single Instruction, Single Data), SIMD (Single Instruction, Multiple Data), MISD (Multiple Instruction, Single Data), and MIMD (Multiple Instruction, Multiple Data). An orange arrow points from the text 'Eksempel: En datamaskin med enkelt prosessor' to the SISD cell. A blue arrow points from the text 'Eksempel: En datamaskin med mange prosessorer' to the MIMD cell. A purple arrow points from the text 'Eksempel: SIMD-instruksjoner' to the SIMD cell.

		Data	
		Single	Multiple
Instruksjoner	Single	SISD	SIMD
	Multiple	MISD	MIMD

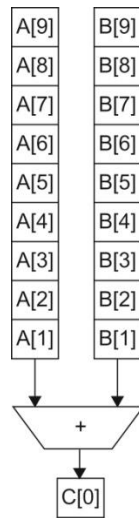
Eksempel: En datamaskin med mange prosessorer

Eksempel:
SIMD-instruksjoner

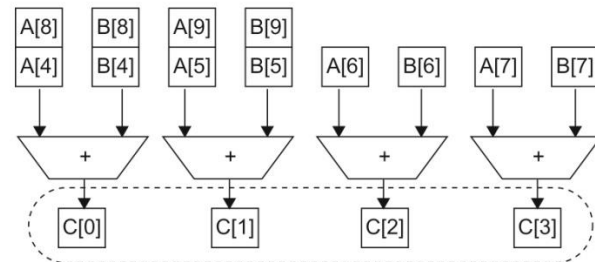
Hvordan utnytte SIMD?

- Fordel: Vi utfører samme instruksjon på mange dataelementer
 - ... så arbeidet gjort per instruksjon øker betraktelig
 - Vi spesifiserer også at elementene er uavhengige av hverandre
- Stort sett alle høyteytelsesprosessorer har i dag SIMD-instruksjoner
 - ... som som gjør samme operasjon på flere dataelementer
- Vi har også vektorprosessorer
 - Disse er typisk mer fleksible i hvordan de støtter vektoroperasjoner (f. eks. variabel vektorlengde)
- Forgreningsinstruksjoner er en utfordring for alle SIMD-arkitekturer

$$C = A + B$$



(a)



(b)

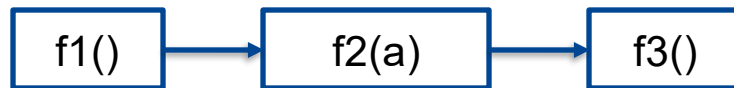


Det store bildet

HVORFOR ER MODERNE DATAMASKINER HETEROGENE?

Hva er nytt?

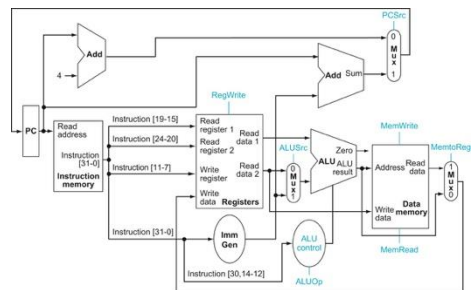
- Vi har så langt kun sett på serielle program
- Vi har nå parallell programvare
... og vi må bygge maskiner som kan utnytte dette



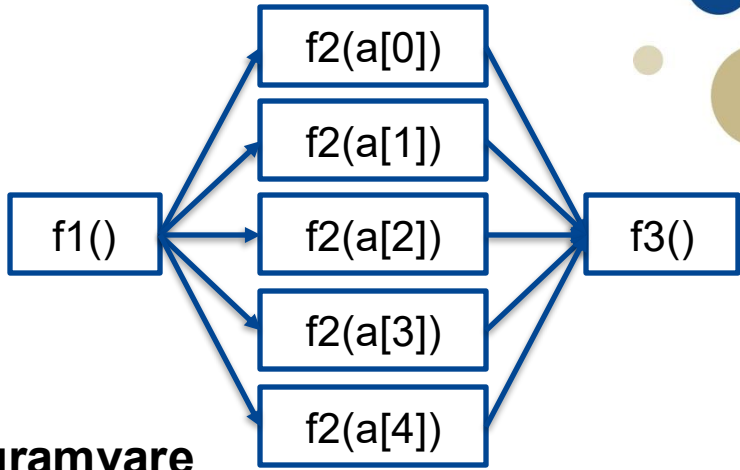
Programvare

Maskinvare

ISA



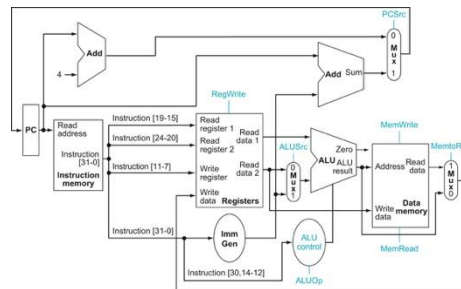
- ... og vi må bygge maskiner som kan utnytte dette



Programvare

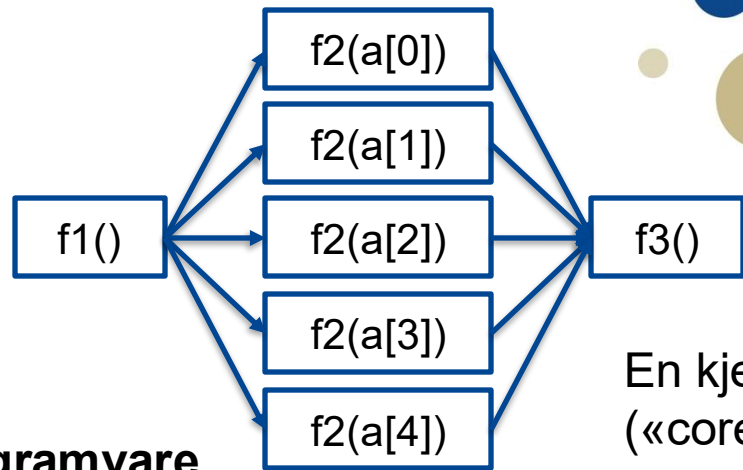
Maskinvare

ISA



Hva er nytt?

- Vi har så langt kun sett på serielle program
- Vi har nå parallell programvare
... og vi må bygge maskiner som kan utnytte dette

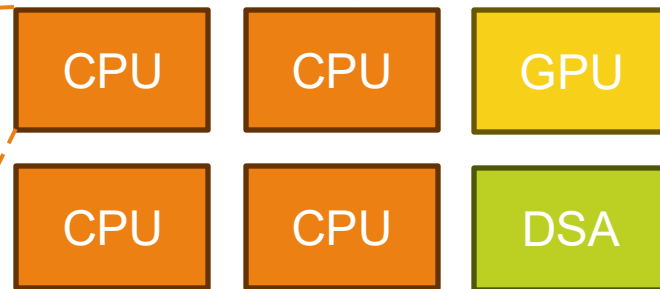
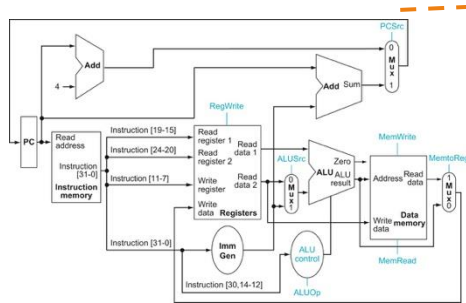


En kjerne
(«core»)

Programvare

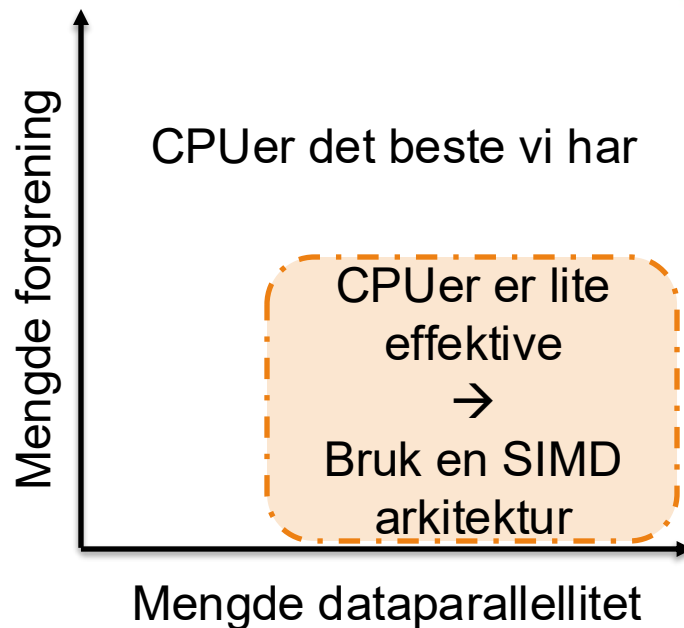
Maskinvare

ISA



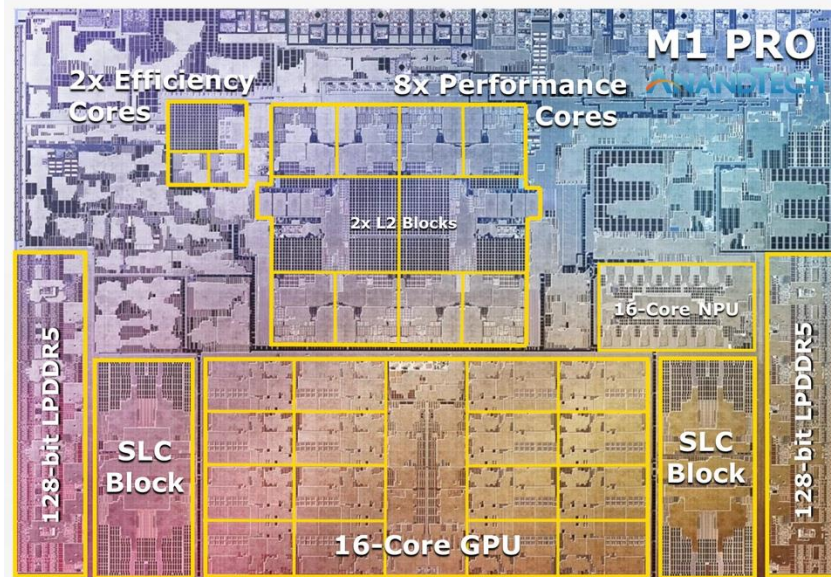
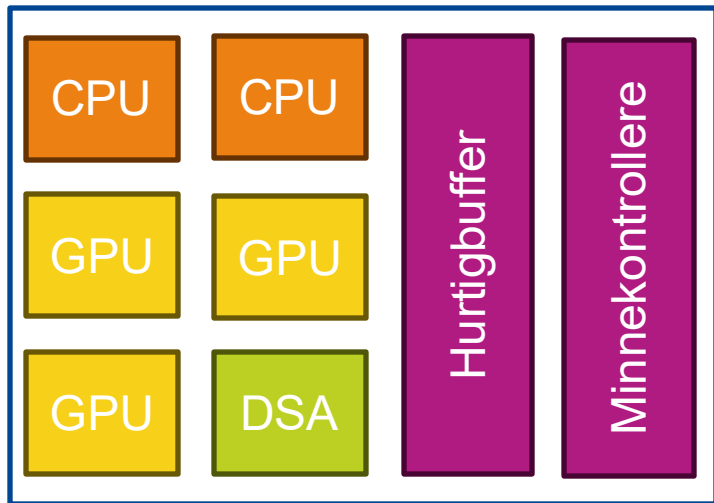
Hva er mulighetsrommet?

- Vi kan lage prosessorkjerner som fokuserer på enten SISD eller SIMD
 - ... og så kobler vi typisk sammen en drøss av dem for å utnytte MIMD
 - (Moderne datamaskiner bruker alle triksene i boka)
- Prosessorene (CPUene) vi har sett på så langt er de mest effektive SISD-arkitekturerne vi kjenner til
 - ... men de er relativt lite effektive for dataparallell kode med lite (eller regulær) forgreining
- Nøkkelobservasjon: **SIMD-arkitekturer er gode på det CPUer er dårlige til**
 - ... og datamaskinen blir dermed mest effektiv hvis vi kombinerer dem
 - Da får vi en heterogen datamaskin



Hvordan ser dette egentlig ut?

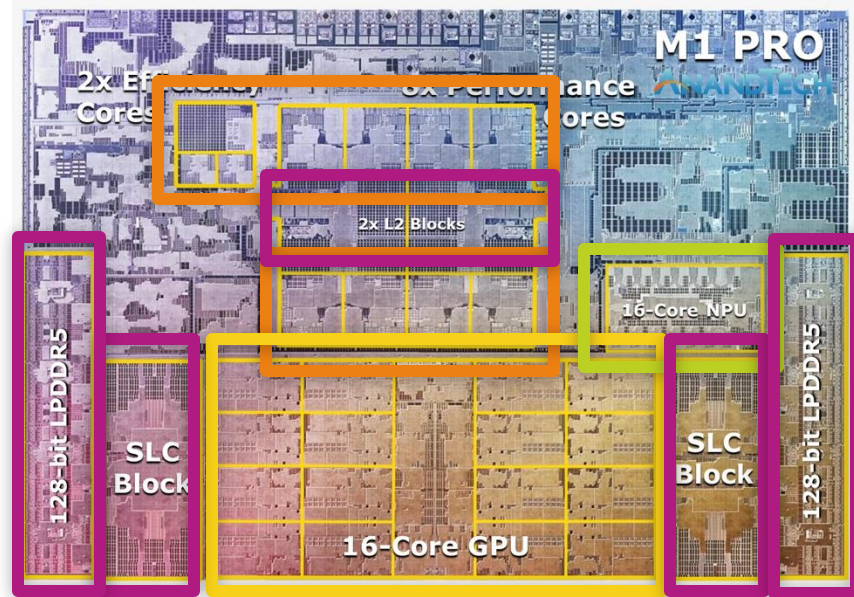
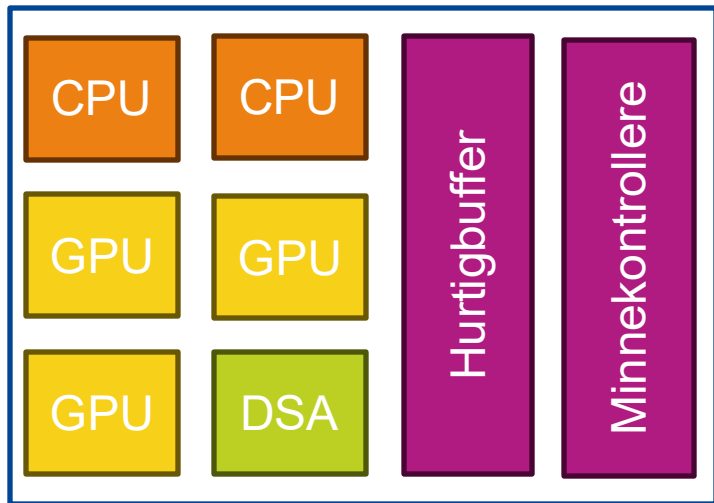
Prosessorbrikken



Prosessorbrikken i min laptop: Apple M1 Pro

Hvordan ser dette egentlig ut?

Proessorbrikken



Proessorbrikken i min laptop: Apple M1 Pro



Tema 7.2 (Kapittel 2.11, 5.10, 6.4, 6.5, 6.8)

MULTIPROSESSORER

Læringsutbytte T7.2

- Studenten kan beskrive den overordnede arkitekturen til **multiprosessorer** med **delt minne** og **distribuert minne** samt kjenne til deres styrker og svakheter.
- Studenten skal kunne beskrive den overordnede arkitekturen til en **flerkjerneprosessor** og kunne motivere for hvorfor arkitekturen er som den er.
- Studenten skal kunne forklare hvorfor **synkronisering** er nødvendig og de grove trekkene i hvordan synkronisering implementeres.
- Studenten skal kjenne til behovet for å holde **hurtigbufferne koherente** og de grove trekkene i hvordan det løses.
- Studenten skal kunne beskrive **minnekonsistensproblemet** og kjenne til de grove trekkene i hvordan det løses.

Hvorfor multiprosessorer?



- Parallellitet mellom uavhengige program er vanlig
- Parallele program med irregulær kontrollflyt er krevende for SIMD-arkitekturer
- Akselererbare applikasjoner trenger også CPUer (Amdahl's lov)
 - ... og fra dette perspektivet er multiprosessoren infrastruktur akseleratorene trenger for å yte best mulig
- Kanskje viktigst: Parallellprogrammering er vanskelig
 - ... men et program for parallelle CPUer kan likne ganske mye på det serielle programmet man allerede har skrevet (se OpenMP eksemplet i boka)

La oss kjøre litt kode

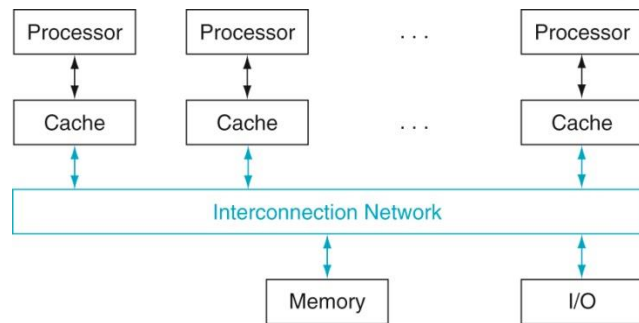
Eksempel: OpenMP



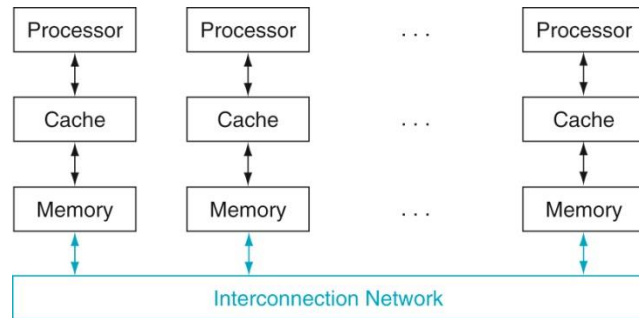
Typer multiprosessorer

- Hvor mange prosessorbrikker er det i systemet?
 - Hvis en: Flerkjerneprosessor («multi-core processor»)
 - Hvis mer enn en: Multiprosessor («multiprocessor»)
 - Alle multiprosessorer i dag er samlinger av flerkjerneprosessorer
- Deler prosessorene det fysiske minnerommet?
 - Ja: Delt minne («Shared Memory Multiprocessor (SMP)»)
 - Nei: Distribuert minne («Distributed-memory multiprocessor»)

Delt minne



Distribuert minne



Flerkjerneprosessorer



- Flerkjerneprosessorer er delt minne arkitekturer
 - ... og deler også typisk siste nivå hurtigbuffer («Last Level Cache (LLC)») mellom kjernene
- Hver prosessorkjerne har typisk to nivåer med private hurtigbuffer
 - Nivå 1 («L1») er nærmest prosessorkjernen og liten (~32KiB) for å få trefftiden ned i ~3 klokkesykler
 - L2 er større (~1MB)
- De private L2 hurtigbufferene er koblet til LLC via et «Network-on-Chip (NoC)»
- Bom i LLC går til minnekontrollerne som tar seg av lesing og skriving til hovedminnet (DRAM)

Eksempel: Synkronisering

Adresse	Verdi
8	7
4	3

Prosess 0

0 → Minnet[4]
Minnet[8] → x1

Prosess 1

1 → Minnet[8]
Minnet[4] → x2

Hvilke verdier har vi i x1 og x2 når koden over har kjørt?



Delt minne: Synkroniseringsinstruksjoner

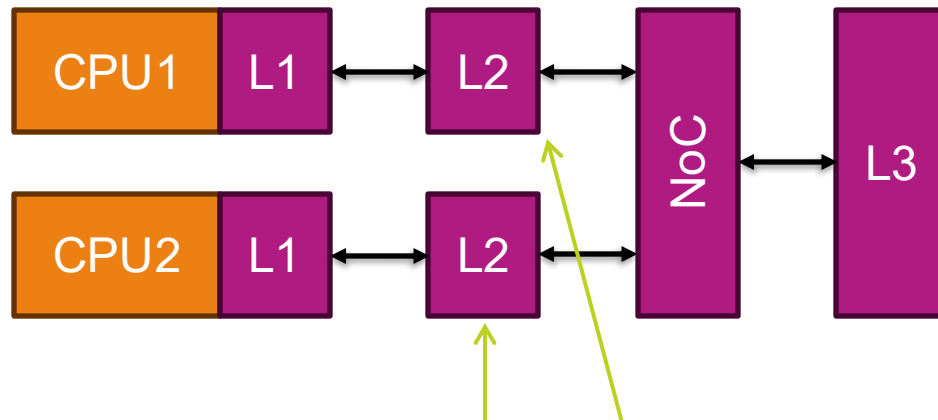


- Enkelte ting må utføres i rekkefølge (synkroniseres)
 - Vi kaller den delen av koden som må utføres i rekkefølge for en kritisk seksjon («critical section»)
 - Eksempel: Når vi legger sammen delsummene fra hver tråd i eksempelprogrammet
- Synkroniseringsmekanismer blir mer effektive hvis de støttes av spesialinstruksjoner
 - Fundamentalt må vi først lese og så skrive en minnelokasjon samt kunne garantere at ingenting har skjedd mellom leseoperasjonen og skriveoperasjonen
 - Typisk implementert med to instruksjoner: «Load reserved» and «store conditional»

Delt minne: Konsistens og koherens



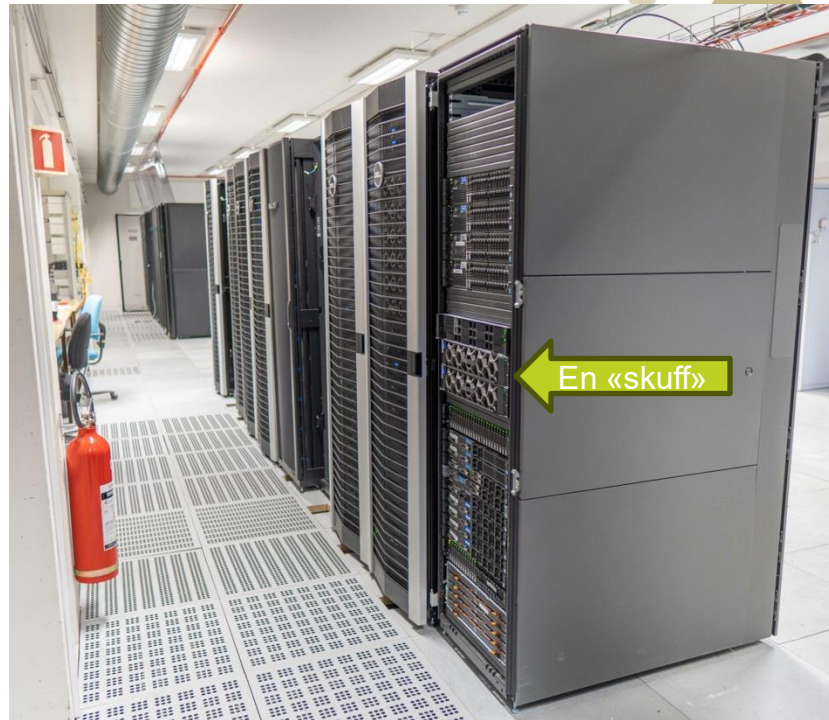
- Koherens («coherence»):
 - Alle prosessorer må se operasjoner på samme dataelement i samme rekkefølge
 - Vi må ha maskinvarestøtte som garanterer denne rekkefølgen («cache coherence protocols»)
 - Vi bryr oss ikke om hvilken rekkefølge dette er så lenge alle er enige
- Konsistens («consistency»)
 - Regler for de lovlige rekkefølgene på operasjoner til ulike dataelement
 - Av alle mulige rekkefølger på operasjoner, er noen lov og noen ikke lov → og maskinen må passe på at bare lovlige rekkefølger oppstår



Hvis en av disse skriver en verdi som den andre allerede har lest, har vi et koherensproblem

Distribuert minne: Clustere

- Hver «skuff» inneholder (flere) delt minne multiprosessorer
 - ... som er koblet sammen med et (kraftig) nettverk
 - Vi kaller hver delt minne multiprosessor en node
- Prosessene på ulike noder kommuniserer ved å sende meldinger over nettverket («message passing»)
- Jo mer applikasjonen kommuniserer mellom nodene, jo kraftigere nettverk trenger vi
 - ... og da begynner vi å nærme oss superdatamaskinene (se <https://top500.org/>)



NTNUs Idun cluster
(~8580 CPU kjerner, 234 GPUer)

Distribuert minne: «Warehouse-Scale Computers»

- Multiprosessorer med distribuert minne kan skaleres så mye man bare vil
 - Googles datasenter i Skien er dimensjonert for et effektforbruk på 860 MW
 - 860 MW i ett år $\rightarrow$ 7.5 TWh
 - Elektrisk energiforbruk for husholdninger i Trøndelag i 2023 $\rightarrow$ 3.5 TWh
- Datasenterapplikasjoner er typisk drevet av forespørsler fra brukere (i global skala)
 - Ditt søk er uavhengig av alle andre søk
 - Datastrukturer som bare leses er enkle å distribuere (i søk: indeksen)



Googles datasenter i Eemshaven, Nederland

«Hardware Multithreading»

- Observasjon: Noen minneintensive applikasjoner utnytter ressursene i prosessorkjernen dårlig
- Løsning: Vi utvider prosessoren slik at den kan kjøre mer enn en prosess av gangen og får dermed flere parallelle minneforespørsler
 - Hver maskinvaretråd må ha sin egen PC og registerfil
 - De andre enhetene i prosessorkjernen kan enten dupliseres eller deles
- Når bytter vi prosess?
 - «Coarse MT»: Når en prosess stanser lenge
 - «Fine MT»: Hver klokkesykel (blant de prosessene som ikke har stanset)
 - «Simultaneous multithreading (SMT)»: Prosessene kjører samtidig
- Man pleier ofte å skru av «hardware multithreading»
 - ... fordi beregningsintensive prosesser vil bli mye tregere

